Sistema de Locadora de Veículos com ASP.NET Core e Entity Framework

Samara Lana da Rocha

[1]Análise e Desenvolvimento de Sistemas – Pontifícia Universidade Católica de Minas Gerais (PUC Minas

samara.lana@sga.pucminas.br

Abstract. *This work presents the development of a backend system for a vehicle rental service using ASP.NET Core, Entity Framework, and SQL Server Express. The system allows the management of customers, vehicles, manufacturers, categories, and rentals, providing complete CRUD operations, data validation, and filters with queries involving multiple tables.*

Resumo. *Este trabalho descreve a implementação de um sistema backend para uma locadora de veículos utilizando ASP.NET Core, Entity Framework e SQL Server Express. O sistema contempla funcionalidades de cadastro, gerenciamento de aluguel e consultas avançadas, garantindo integridade dos dados, organização arquitetural e suporte às principais operações do domínio da locação de veículos.*

1. Introdução

O desenvolvimento de sistemas para locadoras de veículos é importante para automatizar processos operacionais, reduzir erros manuais e melhorar o controle das informações relacionadas ao negócio. Em um cenário de locação, é necessário manter registros confiáveis sobre clientes, veículos, fabricantes, categorias e aluguéis, além de garantir que a disponibilidade dos veículos seja controlada de forma consistente.

Neste contexto, o presente projeto teve como objetivo desenvolver um sistema backend para uma locadora de veículos, utilizando ASP.NET Core, Entity Framework Core e SQL Server Express. A proposta foi construir uma API REST capaz de oferecer operações de cadastro, consulta, atualização e remoção de dados, além de implementar regras de negócio relacionadas ao aluguel e à devolução de veículos.

Entre as principais funcionalidades desenvolvidas, destacam-se o cadastro de clientes, veículos, fabricantes e categorias, o registro de aluguéis e devoluções e a execução de filtros com consultas envolvendo múltiplas tabelas. Como tecnologias principais, foram utilizados C#/.NET, ASP.NET Core, Entity Framework Core, SQL Server Express e Swagger para testes da API.

2. Requisitos Funcionais

O sistema foi projetado para atender às funcionalidades essenciais de uma locadora de veículos. Os requisitos funcionais implementados foram os seguintes:

- Cadastrar fabricantes de veículos;

- Cadastrar categorias de veículos;
- Cadastrar clientes com nome, CPF, e-mail, telefone e data de cadastro;
- Cadastrar veículos com modelo, ano de fabricação, quilometragem, placa, cor, valor da diária, disponibilidade, fabricante e categoria;
- Registrar aluguéis associando um cliente a um veículo em um determinado período;
- Registrar devoluções, informando a quilometragem final e atualizando o valor total da locação;
- Consultar registros por meio de operações CRUD;
- Executar filtros envolvendo mais de uma tabela do banco de dados.

Os principais casos de uso do sistema são:

- **Cadastro de cliente**: permite inserir e consultar clientes da locadora;
- **Cadastro de veículo**: permite inserir veículos vinculados a fabricante e categoria;
- **Registro de aluguel**: associa cliente e veículo, marcando o veículo como indisponível;
- **Registro de devolução**: finaliza o aluguel, calcula o valor total e atualiza a quilometragem do veículo;
- **Consulta com filtros**: permite listar veículos por fabricante, veículos por categoria, aluguéis por cliente, aluguéis em aberto e clientes com quantidade de aluguéis.

3. Arquitetura do Sistema

O sistema foi organizado em uma arquitetura em camadas, com separação de responsabilidades entre os componentes principais da aplicação. Embora não haja interface gráfica, a organização segue uma abordagem semelhante ao padrão MVC, no qual os modelos representam os dados, os controladores concentram a lógica de entrada e saída da API e a camada de persistência realiza a comunicação com o banco de dados.

A estrutura principal do projeto foi dividida da seguinte maneira:

- **Models**: contém as classes de entidade do sistema, como Cliente, Veiculo, Fabricante, CategoriaVeiculo e Aluguel;
- **Data**: contém a classe `LocadoraContext`, responsável pela configuração do Entity Framework e do mapeamento com o banco;
- **Controllers**: contém os endpoints da API REST, implementando as operações CRUD e os filtros;
- **DTOs**: contém classes auxiliares para entrada de dados específicos, como o registro de devolução.

Essa divisão favorece a manutenção do sistema, melhora a organização do código e facilita a identificação das responsabilidades de cada parte da aplicação.

4. Implementação Técnica

A implementação foi realizada em C# com ASP.NET Core, utilizando o Entity Framework Core como tecnologia de mapeamento objeto-relacional e o SQL Server Express como banco de dados relacional. Para testes dos endpoints, foi utilizado o Swagger.

As principais tecnologias empregadas foram:

- C#/.NET;
- ASP.NET Core Web API;
- Entity Framework Core;
- SQL Server Express;
- Swagger/OpenAPI.

As principais classes do sistema possuem as seguintes responsabilidades:

- **Cliente**: representa o cliente da locadora;
- **Fabricante**: representa a marca do veículo;
- **CategoriaVeiculo**: representa a classificação do veículo;
- **Veiculo**: representa os veículos disponíveis para locação;
- **Aluguel**: representa a locação de um veículo por um cliente;
- **LocadoraContext**: configura o banco de dados, os relacionamentos e as restrições de integridade.

Em termos de design, foi adotado o uso de Data Annotations para validação básica de atributos, como obrigatoriedade de campos, tamanho de strings, faixas de valores numéricos e formato de e-mail. Além disso, foram configurados índices únicos para CPF, e-mail, placa e nome da categoria, reforçando a consistência dos dados. Também foi utilizado um enum para representar o status do aluguel, permitindo maior controle semântico sobre os estados possíveis da locação.

5. Descrição do Banco de Dados

O sistema foi desenvolvido com persistência em banco de dados relacional utilizando SQL Server Express, sendo o mapeamento objeto-relacional realizado com o Entity Framework Core.

O modelo de dados foi estruturado para atender às necessidades de uma locadora de veículos, contemplando o cadastro de fabricantes, categorias, clientes, veículos e aluguéis. O banco foi projetado com foco em integridade referencial, organização dos relacionamentos e suporte às regras de negócio implementadas na API.

5.1. Tabelas Principais

As tabelas principais do sistema são:

- **Fabricantes**: armazena as marcas dos veículos cadastrados, contendo informações como nome e país de origem;
- **CategoriasVeiculo**: armazena a classificação dos veículos, como SUV, Sedan e Hatch;
- **Clientes**: armazena os dados dos clientes da locadora, incluindo nome, CPF, e-mail, telefone e data de cadastro;
- **Veiculos**: armazena os dados dos veículos disponíveis para locação, como modelo, ano de fabricação, quilometragem atual, placa, cor, valor da diária e disponibilidade;
- **Alugueis**: registra as operações de locação, vinculando cliente e veículo em um determinado período, além de armazenar quilometragem inicial, quilometragem final, valor da diária, valor total e status do aluguel.

O banco também possui a tabela técnica **__EFMigrationsHistory**, criada automaticamente pelo Entity Framework, responsável por armazenar o histórico das migrations aplicadas ao banco.

5.2. Relacionamentos

Os relacionamentos entre as entidades foram modelados da seguinte forma:

- Um **fabricante** pode estar associado a vários veículos, enquanto cada veículo pertence a apenas um fabricante;
- Uma **categoria** pode classificar vários veículos, enquanto cada veículo pertence a apenas uma categoria;
- Um **cliente** pode realizar vários aluguéis ao longo do tempo;
- Um **veículo** pode participar de vários aluguéis, desde que em períodos diferentes;
- Cada **aluguel** está obrigatoriamente associado a um cliente e a um veículo.

5.3. Restrições de Integridade

Para garantir consistência dos dados, foram adotadas algumas restrições importantes no banco:

- CPF do cliente com índice único;
- E-mail do cliente com índice único;
- Placa do veículo com índice único;
- Nome da categoria com índice único;
- Relacionamentos com exclusão restrita, evitando remoção de registros ainda referenciados;
- Conversão do status do aluguel para texto, facilitando a leitura no banco.

Essas restrições foram configuradas no método `OnModelCreating` da classe `LocadoraContext`, utilizando recursos do Entity Framework Core.

5.4. Exemplos de Consultas SQL

A seguir, são apresentados exemplos de consultas que representam operações importantes no sistema.

Consulta de veículos com seus fabricantes:

```
SELECT v.Modelo, v.Placa, f.Nome AS Fabricante
FROM Veiculos v
JOIN Fabricantes f ON v.FabricanteId = f.Id;
```

Consulta de veículos com suas categorias:

```
SELECT v.Modelo, c.Nome AS Categoria
FROM Veiculos v
JOIN CategoriasVeiculo c ON v.CategoriaVeiculoId = c.Id;
```

Consulta de aluguéis com dados de cliente e veículo:

```
SELECT a.Id, cli.Nome AS Cliente, v.Modelo AS Veiculo,
       a.DataInicio, a.DataFimPrevista, a.Status
FROM Alugueis a
JOIN Clientes cli ON a.ClienteId = cli.Id
JOIN Veiculos v ON a.VeiculoId = v.Id;
```

Consulta de clientes com ou sem aluguel:

```
SELECT c.Nome, c.Email, COUNT(a.Id) AS QuantidadeAlugueis
FROM Clientes c
LEFT JOIN Alugueis a ON c.Id = a.ClienteId
GROUP BY c.Nome, c.Email;
```

6. Testes e Validação

Foram realizados testes manuais com os principais endpoints da API utilizando o Swagger. Os testes tiveram como objetivo validar tanto as operações básicas de persistência quanto as regras de negócio implementadas no sistema.

Os testes abrangeram:

- Inserção de dados por meio de operações POST;
- Consulta de dados por meio de operações GET;
- Atualização de dados por meio de operações PUT;
- Exclusão de registros por meio de operações DELETE;
- Validação de erros e restrições, como tentativa de cadastro com CPF duplicado;
- Fluxo de aluguel e devolução de veículos.

Durante os testes, foi verificado que a aplicação validava corretamente restrições de unicidade, como no caso de clientes com CPF já cadastrado. Também foram testadas regras específicas de negócio, como o cálculo do valor total do aluguel, a atualização da quilometragem do veículo e a alteração de disponibilidade após a devolução.

Os testes realizados foram predominantemente manuais. Não foram implementados testes automatizados nesta etapa do projeto, de forma que a cobertura concentrou-se na validação funcional dos principais endpoints e fluxos do sistema. Ainda assim, os testes executados foram suficientes para comprovar o funcionamento das operações centrais da aplicação.

Inserção de dados (POST)

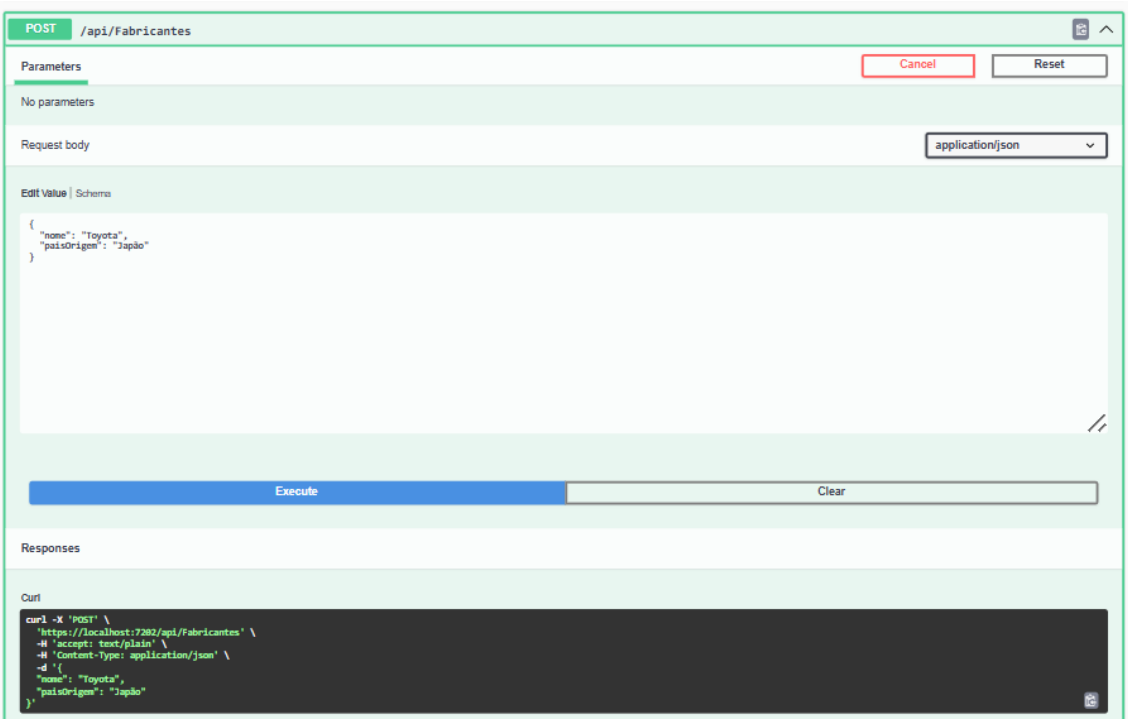

Figura 1. Teste POST Fabricante parte 1

Request URL

https://localhost:7202/api/Fabricantes

Server response

Code	Details
201 Undocumented	Response body

```
{
  "id": 2,
  "nome": "Toyota",
  "paisOrigem": "Japão",
  "veiculos": []
}
```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Sun,19 Apr 2026 23:53:16 GMT
location: https://localhost:7202/api/Fabricantes/2
server: Kestrel
```

Responses

Code	Description	Links
200	OK	No links

Media type

text/plain

Controls Accept header.

Example Value | Schema

```
{
  "id": 0,
  "nome": "string",
  "paisOrigem": "string",
  "veiculos": [
    "string"
  ]
}
```

Figura 2. Teste POST Fabricante parte 2

POST /api/CategoriasVeiculo

Parameters

Cancel Reset

No parameters

Request body

application/json

Edit Value | Schema

```
{
  "nome": "SUV",
  "descricao": "Veículos utilitários esportivos"
}
```

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'https://localhost:7202/api/CategoriasVeiculo' \
  -H 'accept: text/plain' \
  -H 'Content-Type: application/json' \
  -d '{
  "nome": "SUV",
  "descricao": "Veículos utilitários esportivos"
}'
```

Figura 3. Teste POST Categoria do Veículo parte 1

Request URL

https://localhost:7202/api/CategoriasVeiculo

Server response

Code | Details

201 Undocumented — Response body

```
{
  "id": 3,
  "nome": "SUV",
  "descricao": "Veículos utilitários esportivos",
  "veiculos": []
}
```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Sun,19 Apr 2026 23:56:57 GMT
location: https://localhost:7202/api/CategoriasVeiculo/3
server: Kestrel
```

Responses

Code	Description	Links
200	OK Media type: text/plain Controls Accept header. Example Value | Schema	No links

```
{
  "id": 0,
  "nome": "string",
  "descricao": "string",
  "veiculos": [
    "string"
  ]
}
```

Figura 4. Teste POST Categoria do Veículo parte 2

Validação de dados (CPF duplicado)

Durante os testes, foi realizada uma tentativa de cadastro de cliente com CPF já existente, resultando em erro, demonstrando o funcionamento correto da validação.

POST /api/Clientes

Parameters — Cancel — Reset

No parameters

Request body — application/json

Edit Value | Schema

```
{
  "nome": "Gabriel Santos",
  "cpf": "12345678901",
  "email": "gabriel@email.com",
  "telefone": "11999999999",
  "dataCadastro": "2026-04-11T10:00:00"
}
```

Execute — Clear

Responses

Curl

```
curl -X 'POST' \
  'https://localhost:7202/api/Clientes' \
  -H 'accept: text/plain' \
  -H 'Content-Type: application/json' \
  -d '{
  "nome": "Gabriel Santos",
  "cpf": "12345678901",
  "email": "gabriel@email.com",
  "telefone": "11999999999",
  "dataCadastro": "2026-04-11T10:00:00"
}'
```

Request URL

https://localhost:7202/api/Clientes

Figura 5. Teste POST Cliente com CPF duplicado parte 1

Request URL

https://localhost:7202/api/Clientes

Server response

Code	Details
400 Undocumented	Error: response status is 400

Response body

```
{
  "mensagem": "CPF já cadastrado."
}
```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Sun,19 Apr 2026 23:59:46 GMT
server: Kestrel
```

Responses

Code	Description	Links
200	OK Media type: text/plain Controls Accept header. Example Value | Schema	No links

```
{
  "id": 0,
  "nome": "string",
  "cpf": "stringstrin",
  "email": "user@example.com",
  "telefone": "string",
  "dataCadastro": "2026-04-20T00:00:27.285Z",
  "alugueis": [
    "string"
  ]
}
```

Figura 6. Teste POST Cliente com CPF duplicado parte 2

Consultas (GET)

GET /api/Clientes

Parameters

Cancel

No parameters

Execute | Clear

Responses

Curl

```
curl -X 'GET' \
  'https://localhost:7202/api/Clientes' \
  -H 'accept: text/plain'
```

Request URL

https://localhost:7202/api/Clientes

Server response

Code	Details
200	

Response body

```
[
  {
    "id": 1,
    "nome": "Gabriel Santos",
    "cpf": "12345678901",
    "email": "gabriel@email.com",
    "telefone": "119999999",
    "dataCadastro": "2026-04-11T13:35:15.81",
    "alugueis": []
  }
]
```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Mon,20 Apr 2026 00:04:18 GMT
server: Kestrel
```

Figura 7. Teste GET Clientes

GET /api/Veiculos

Parameters

Cancel

No parameters

Execute | Clear

Responses

Curl

```
curl -X 'GET' \
  'https://localhost:7202/api/Veiculos' \
  -H 'accept: */*'
```

Request URL

https://localhost:7202/api/Veiculos

Server response

Code | Details

200

Response body


```
[
  {
    "id": 1,
    "modelo": "Corolla Cross",
    "anoFabricacao": 2023,
    "quilometragemAtual": 15000,
    "placa": "ABC1234",
    "cor": "Prata",
    "valorDiariaBase": 180,
    "disponivel": true,
    "fabricante": "Toyota",
    "categoria": "SUV"
  }
]
```


Download

Response headers

```
content-type: application/json; charset=utf-8
date: Mon,20 Apr 2026 00:15:42 GMT
server: Kestrel
```

Figura 8. Teste GET Veículo

Cadastro completo

POST /api/Clientes

Parameters

Cancel | Reset

No parameters

Request body

application/json

Edit Value | Schema


```
{
  "nome": "Samara Santos",
  "cpf": "12345678902",
  "email": "samara@email.com",
  "telefone": "11999999998",
  "dataCadastro": "2026-04-19T10:00:00"
}
```


Execute | Clear

Responses

Curl

```
curl -X 'POST' \
  'https://localhost:7202/api/Clientes' \
  -H 'accept: text/plain' \
  -H 'Content-Type: application/json' \
  -d '{
  "nome": "Samara Santos",
  "cpf": "12345678902",
  "email": "samara@email.com",
  "telefone": "11999999998",
  "dataCadastro": "2026-04-19T10:00:00"
```

Figura 9. Teste POST Cliente parte 1

Request URL

https://localhost:7202/api/Clientes

Server response

Code	Details
201 Undocumented	Response body

```
{
  "id": 2,
  "nome": "Samara Santos",
  "cpf": "12345678902",
  "email": "samara@email.com",
  "telefone": "11999999998",
  "dataCadastro": "2026-04-19T10:00:00",
  "alugueis": []
}
```

Download

Response headers

```
content-type: application/json; charset=utf-8
date: Mon,20 Apr 2026 00:07:57 GMT
location: https://localhost:7202/api/Clientes/2
server: Kestrel
```

Responses

Code	Description	Links
200	OK Media type: text/plain Controls Accept header. Example Value | Schema	No links

```
{
  "id": 0,
  "nome": "string",
  "cpf": "stringstrin",
  "email": "user@example.com",
  "telefone": "string",
  "dataCadastro": "2026-04-20T00:07:57.368Z",
  "alugueis": [
    "string"
  ]
}
```

Figura 10. Teste POST Cliente parte 2

POST /api/Veiculos

Parameters

Cancel Reset

No parameters

Request body application/json

Edit Value | Schema

```
{
  "modelo": "UNO",
  "anoFabricacao": 2023,
  "quilometragemAtual": 15000,
  "placa": "ABC1235",
  "cor": "Prata",
  "valorDiariaBase": 170.00,
  "disponivel": true,
  "fabricanteId": 1,
  "categoriaVeiculoId": 1
}
```

Execute Clear

Responses

Curl

```
curl -X 'POST' \
  'https://localhost:7202/api/Veiculos' \
  -H 'accept: text/plain' \
  -H 'Content-Type: application/json' \
  -d '{
  "modelo": "UNO",
  "anoFabricacao": 2023,
  "quilometragemAtual": 15000,
  "placa": "ABC1235",
  "cor": "Prata",
  "valorDiariaBase": 170.00,
  "disponivel": true,
  "fabricanteId": 1,
  "categoriaVeiculoId": 1
}'
```

Figura 11. Teste POST Veículo

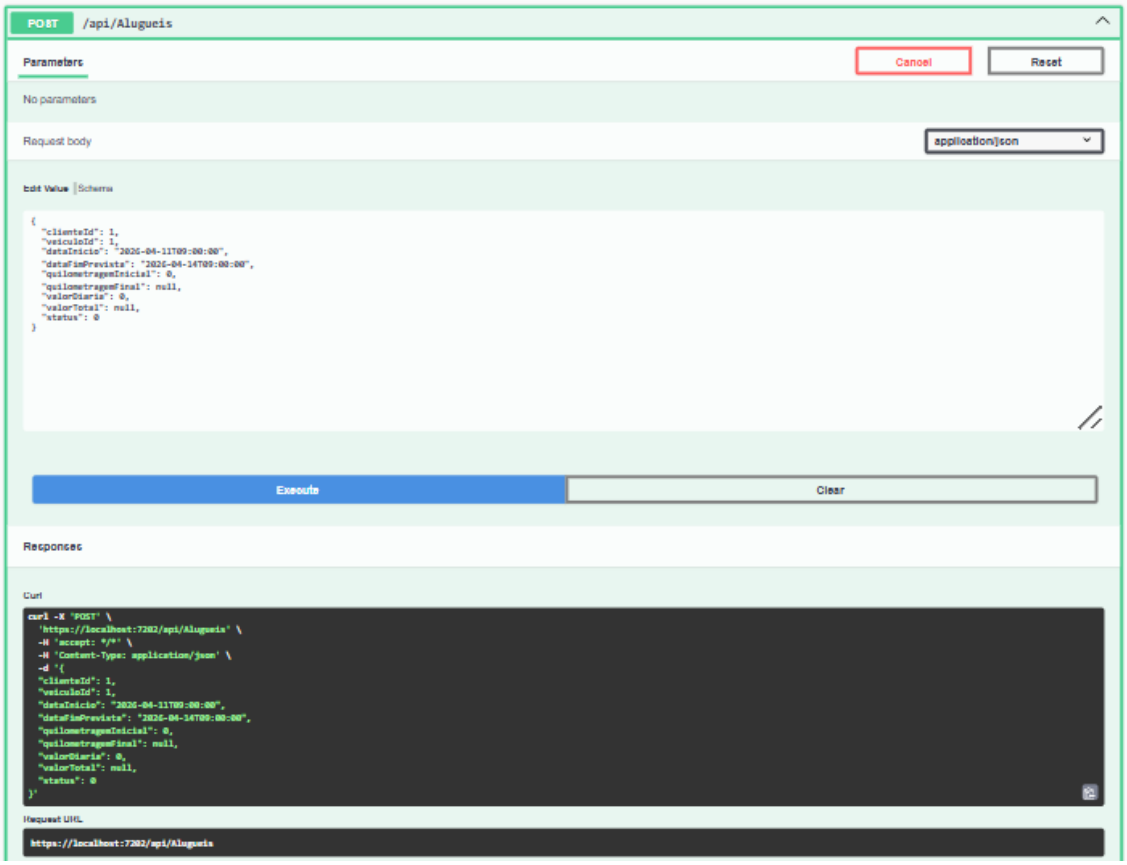

Figura 12. Teste POST Aluguel

Devolução de veículo

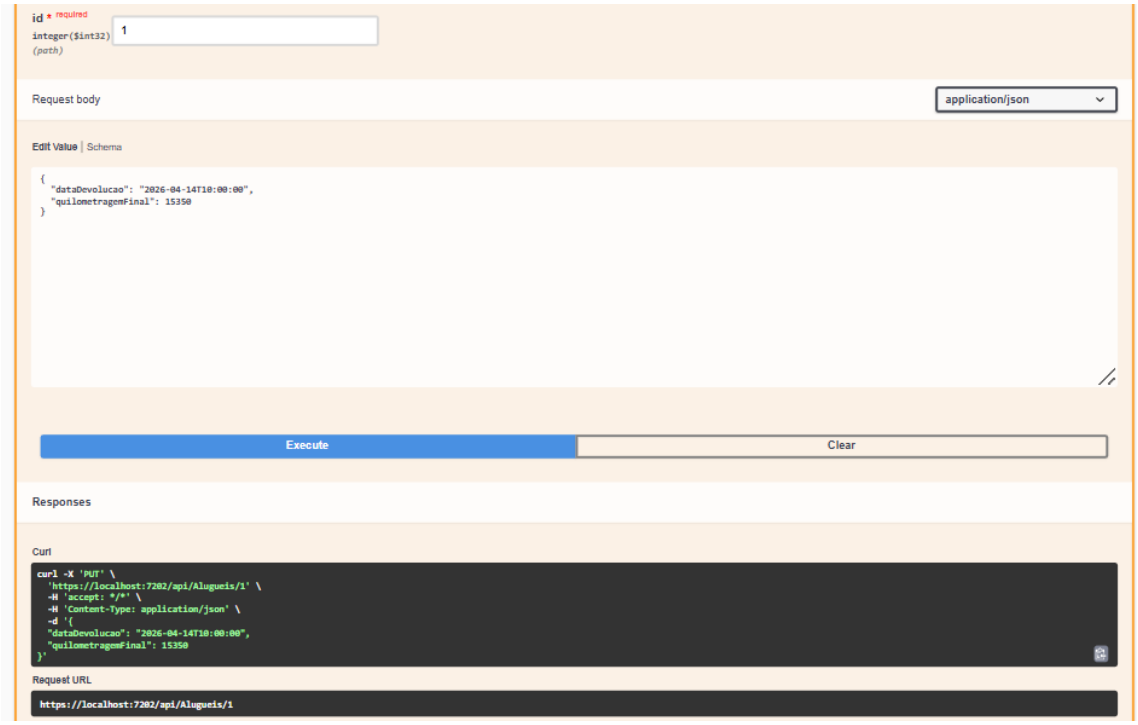

Figura 13. Teste PUT Devolução de Aluguel

Operações adicionais

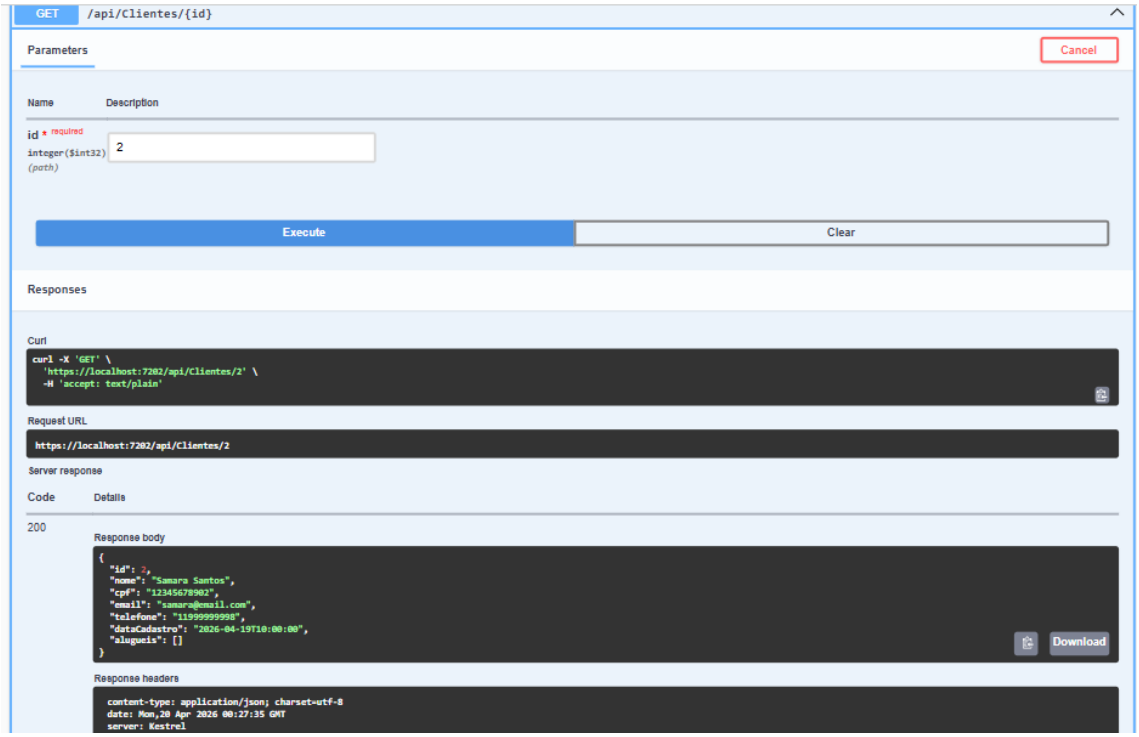

Figura 14. Teste GET Cliente por ID

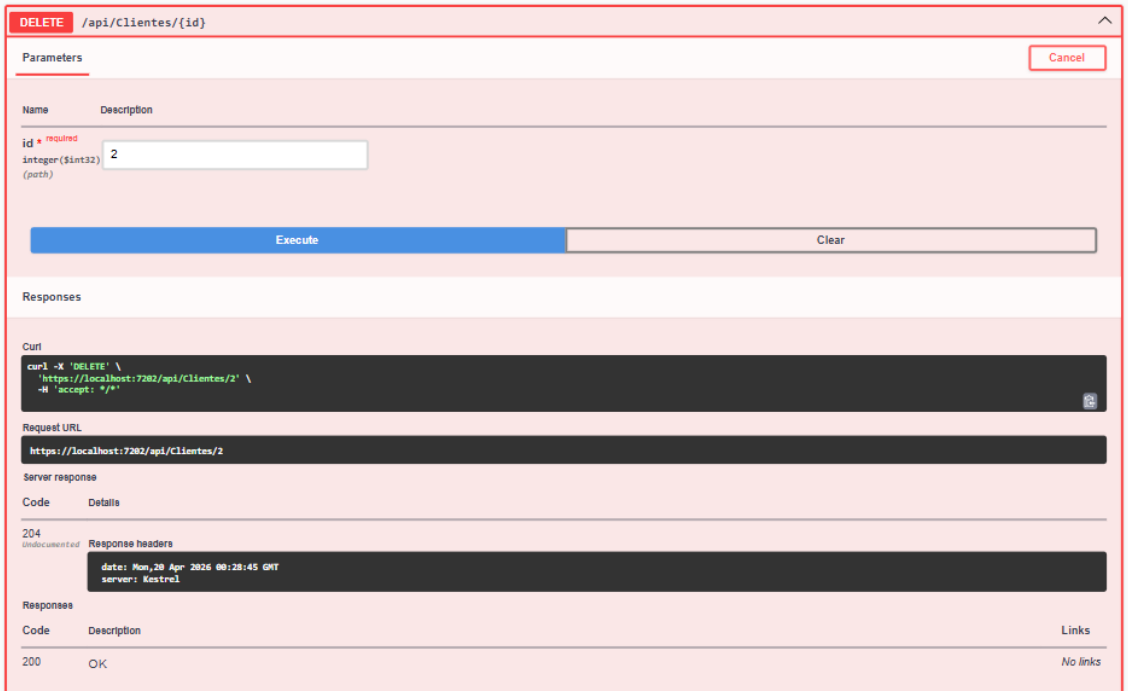

Figura 15. Teste DELETE Cliente

7. Resultados e Conclusões

Os resultados obtidos demonstraram que o sistema atendeu aos requisitos funcionais definidos para o projeto. Foi possível verificar o funcionamento correto dos endpoints de cadastro, consulta, atualização e exclusão, bem como a persistência adequada das informações no banco de dados.

Também foi confirmada a conformidade com requisitos importantes, como o uso do Entity Framework com SQL Server Express, a validação de dados de entrada, a existência de filtros com consultas envolvendo múltiplas tabelas e a implementação das principais regras de negócio do domínio da locadora.

Conclui-se, portanto, que o sistema desenvolvido foi eficaz em atender às necessidades propostas para uma locadora de veículos em nível acadêmico, oferecendo uma base funcional, organizada e coerente com os objetivos do trabalho.

8. Considerações Finais

O desenvolvimento deste projeto proporcionou aprendizado prático em modelagem de banco de dados, desenvolvimento de APIs REST, uso do Entity Framework Core, testes com Swagger e organização de um projeto backend em camadas.

Além do ganho técnico, a experiência contribuiu para o entendimento da integração entre banco de dados e aplicação, bem como para a aplicação de conceitos estudados em sala em um contexto prático.

Como melhorias futuras, destacam-se:

- Desenvolvimento de interface gráfica para interação com o sistema;

- Implementação de autenticação e controle de acesso;
- Inclusão de relatórios gerenciais;
- Expansão da cobertura com testes automatizados.

9. Referências

MICROSOFT. *ASP.NET Core Documentation*. Disponível em: `https://learn.microsoft.com/aspnet/core`. Acesso em: 10 abr. 2026.

MICROSOFT. *Entity Framework Core Documentation*. Disponível em: `https://learn.microsoft.com/ef/core`. Acesso em: 10 abr. 2026.

MICROSOFT. *SQL Server Documentation*. Disponível em: `https://learn.microsoft.com/sql`. Acesso em: 10 abr. 2026.

SMARTBEAR. *Swagger Documentation*. Disponível em: `https://swagger.io/docs`. Acesso em: 10 abr. 2026.